# Mid-Point Research Summary Report

**Phases A-B: From Physical AI Landscape Review to Rover Fleet Anomaly Benchmark**

Xirui (Crissy) Chen

Applied AI Analyst Intern

## Executive Summary

During the first four weeks, I moved from strategic orientation to a reproducible anomaly-detection benchmark. Phase A established the physical AI and competitive context. Phase B converted that context into a telemetry analytics pipeline using Aido Rover as the data substrate and Sentinel-style fleet monitoring as the operating frame. The midpoint result is not just a set of weekly artifacts. It is an evidence chain: landscape research, competitor positioning, synthetic telemetry generation, feature engineering, controlled fault injection, multi-method benchmarking, statistical testing, and an operating-regime recommendation.

The Week 4 benchmark used the 5-unit, 1-day Week 3 sample with 432,000 rows at 1 Hz. I injected 150 controlled fault events across five fault types and evaluated four methods across 10 seeded train/test splits. LOF produced the strongest overall accuracy profile, with AUROC 0.985 ± 0.008 and F1 0.651 ± 0.055. One-Class SVM was a close challenger with AUROC 0.981 ± 0.007 and F1 0.617 ± 0.075. Isolation Forest remained useful as a lightweight screen, but its overall AUROC was lower at 0.844 ± 0.044. The LSTM autoencoder did not justify its added runtime and tuning cost in the current benchmark, with AUROC 0.848 ± 0.037.

My recommendation is to adopt LOF as the primary accuracy path for offline review, fleet-health triage, and analyst dashboards; keep One-Class SVM as a challenger model, especially for software-hang-like faults; and use Isolation Forest only as a latency-constrained first-pass screen with explicit rule-based guardrails for battery, GPS, stale telemetry, and task-success degradation.

## 1. Research Objective and Phase A-B Context

The midpoint review covers two phases. Phase A, Weeks 1-2, built the strategic and product context through a physical AI landscape brief, PIC 2.0 conceptual map, competitive vendor matrix, and strategic gap memo. Phase B, Weeks 3-4, turned that context into a technical workflow: synthetic Rover telemetry, exploratory analysis, feature engineering, and anomaly detection benchmarking.

The central question for the midpoint is practical rather than abstract: which anomaly-detection method should become the default path for Rover and Sentinel-style fleet health monitoring? This question connects the competitive intelligence work to a real analytics design choice. Rover

provides the telemetry-rich substrate, while Sentinel provides the operational framing around alerts, false positives, uncertainty, and escalation discipline.

## 2. Dataset and Telemetry Substrate

The Week 3 generator creates synthetic Aido Rover fleet telemetry at 1 Hz. The default evaluation dataset contains five Rover units over one day, producing 432,000 rows. The full generator configuration supports a 50-unit, 30-day horizon, which would produce 129.6 million rows. The benchmark uses the smaller 5-unit sample so the full pipeline can be run locally and audited during the midpoint review.

| | |
|---|---|
| Telemetry source | Synthetic Aido Rover fleet telemetry |
| Default benchmark sample | 5 units x 1 day x 1 Hz |
| Rows in default sample | 432,000 |
| Partitions | 5 robot-day files |
| Full-scale generator capacity | 50 units x 30 days x 1 Hz = 129,600,000 rows |
| Primary signals | Joint angles, IMU accel and gyro, motor current per drive, battery SoC and voltage, WiFi RSSI, GPS fix quality, task-success flags |
| Engineered feature layer | 25 features including rolling statistics, cross-sensor ratios, and inter-unit z-scores |

*Table 1. Dataset and telemetry substrate used for the Week 4 benchmark.*

The feature layer was important because the goal was not only to detect mathematical outliers. The benchmark needed features that carried operational meaning. For example, voltage-versus-SoC residuals help identify power anomalies; speed-versus-current efficiency captures whether the robot is moving normally for its propulsion load; and fleet-relative z-scores help flag units that behave differently from peers at the same timestamp.

## 3. Fault Injection Design

The benchmark injected faults only into held-out test blocks after the train/test split. This was a deliberate leakage-control decision. If faults were injected before splitting, a model could indirectly learn properties of the injected anomalies during preprocessing or normalization. In the final design, all training windows remain clean, scalers are fit only on clean training windows, and faults appear only in held-out evaluation windows.

Each split includes five controlled fault types: motor stall, gradual sensor drift, accelerated battery degradation, GPS jitter with spoofing-like behavior, and intermittent software hang. Across 10 splits, the benchmark produced 150 fault events, with 30 events per fault type. Fault durations generally ranged from roughly four to fifteen minutes, which is long enough for one-minute

operational windows to capture the pattern but still short enough to resemble localized field incidents.

| Fault type | Events | Mean duration (s) | Duration range (s) | Mean severity |
|---|---|---|---|---|
| Motor stall | 30 | 587.6 | 258-883 | 0.802 |
| Sensor drift | 30 | 588.6 | 265-893 | 0.818 |
| Battery degradation | 30 | 618.5 | 275-898 | 0.791 |
| GPS jitter | 30 | 581.9 | 269-900 | 0.816 |
| Software hang | 30 | 528.4 | 248-853 | 0.822 |

*Table 2. Controlled fault injection summary across 10 seeded splits.*

## 4. Benchmark Methods and Evaluation Protocol

I benchmarked four unsupervised anomaly detection methods: Isolation Forest, One-Class SVM, Local Outlier Factor, and an LSTM autoencoder. The classical methods were trained on clean one-minute training windows. The LSTM autoencoder used short sequences of window-level features and scored reconstruction error. All methods were evaluated on the same split seeds and fault-injection protocol.

The primary metrics were precision, recall, F1, and AUROC. Precision is especially relevant to alert burden because low precision means operators see more false alerts. Recall matters for safety and monitoring because missed faults are costly. F1 summarizes the thresholded precision-recall tradeoff. AUROC evaluates ranking quality from continuous anomaly scores and is useful for comparing methods before selecting a final operating threshold.

The benchmark reports mean and standard deviation across 10 seeded splits. I also ran paired Wilcoxon signed-rank tests on AUROC differences because each method was evaluated on the same split seeds. This paired design is closer to backtesting discipline than a single-run comparison and helps avoid over-interpreting one favorable random split.

## 5. Overall Benchmark Results

| Method | Precision | Recall | F1 | AUROC |
|---|---|---|---|---|
| Isolation Forest | 0.373 ± 0.129 | 0.394 ± 0.096 | 0.365 ± 0.074 | 0.844 ± 0.044 |
| One-Class SVM | 0.458 ± 0.080 | 0.966 ± 0.022 | 0.617 ± 0.075 | 0.981 ± 0.007 |
| LOF | 0.492 ± 0.060 | 0.970 ± 0.033 | 0.651 ± 0.055 | 0.985 ± 0.008 |
| LSTM Autoencoder | 0.382 ± 0.082 | 0.624 ± 0.069 | 0.466 ± 0.063 | 0.848 ± 0.037 |

*Table 3. Overall benchmark metrics, reported as mean ± standard deviation across 10 seeded splits.*

LOF had the strongest overall profile. It achieved the highest AUROC and F1 while maintaining very high recall. One-Class SVM was close in AUROC and recall, which makes it a credible challenger. The practical difference is that LOF produced a slightly better thresholded F1 in this benchmark, while One-Class SVM remained competitive enough that I would not discard it before testing the full 50-unit, 30-day dataset.

Isolation Forest was materially weaker in accuracy but still has an operational role. Its value is not that it is the most accurate method. Its value is that it can serve as a simpler, lightweight first-pass screen. The LSTM autoencoder was less convincing in this run because it added complexity without improving AUROC or F1.

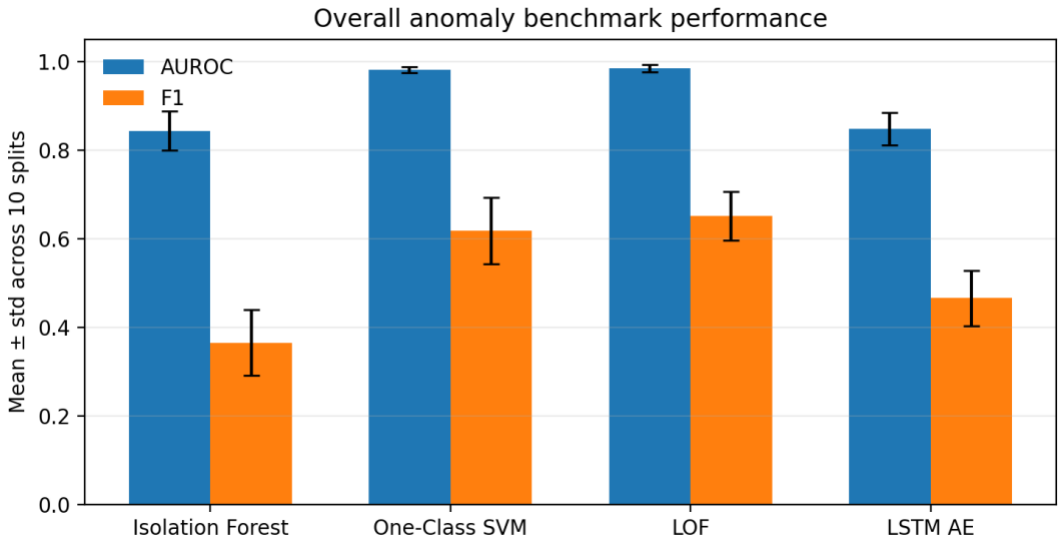


*Figure 1. Overall AUROC and F1 by method across 10 seeded splits.*

## 6. Per-Fault Results

The per-fault results show why the final recommendation should not rely only on the overall average. LOF ranked best or effectively best on most fault types, but One-Class SVM was very close and slightly stronger for software-hang-like behavior by AUROC. Motor stall was detected well by nearly all methods, while accelerated battery degradation and GPS jitter created a clearer separation between LOF/One-Class SVM and the other methods.

| Fault type | Isolation Forest | One-Class SVM | LOF | LSTM AE |
|---|---|---|---|---|
| Motor stall | 0.965 ± 0.029 | 0.997 ± 0.001 | 0.998 ± 0.001 | 0.986 ± 0.011 |
| Sensor drift | 0.867 ± 0.064 | 0.991 ± 0.010 | 0.992 ± 0.008 | 0.978 ± 0.013 |
| Battery degradation | 0.763 ± 0.109 | 0.979 ± 0.031 | 0.982 ± 0.028 | 0.566 ± 0.128 |
| GPS jitter | 0.753 ± 0.085 | 0.947 ± 0.010 | 0.956 ± 0.017 | 0.857 ± 0.028 |
| Software hang | 0.870 ± 0.070 | 0.996 ± 0.002 | 0.996 ± 0.001 | 0.890 ± 0.053 |

*Table 4. Per-fault AUROC, reported as mean ± standard deviation across 10 seeded splits.*

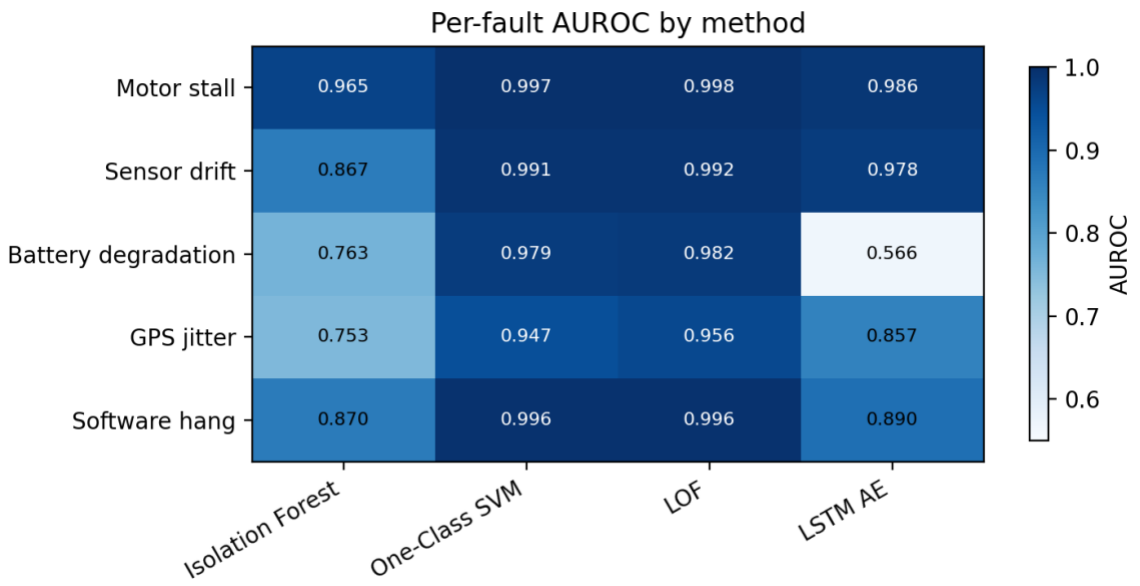


*Figure 2. Per-fault AUROC heatmap. Darker cells indicate stronger ranking performance.*

The most important per-fault pattern is that LOF and One-Class SVM consistently dominated Isolation Forest and the LSTM autoencoder on gradual and consistency-based anomalies. For accelerated battery degradation, LOF reached AUROC 0.982 ± 0.028 compared with 0.763 ± 0.109 for Isolation Forest and 0.566 ± 0.128 for the LSTM autoencoder. For GPS jitter, LOF reached 0.956 ± 0.017 compared with 0.753 ± 0.085 for Isolation Forest. These are the types of faults where cross-sensor consistency and neighborhood structure matter.

## 7. Statistical Comparison

The paired Wilcoxon tests support the main recommendation but also show where the recommendation should be cautious. Overall, LOF beat Isolation Forest by +0.141 AUROC with p = 0.0020 and beat the LSTM autoencoder by +0.137 AUROC with p = 0.0020. LOF also beat One-Class SVM by +0.003 AUROC with p = 0.0195. However, that last gap is small in operational terms, so I interpret it as evidence to prefer LOF as the default accuracy path, not as evidence to eliminate One-Class SVM.

| Comparison | Mean AUROC difference | Wilcoxon p-value | n paired splits |
|---|---|---|---|
| LOF vs. Isolation Forest | +0.141 | 0.0020 | 10 |
| LOF vs. One-Class SVM | +0.003 | 0.0195 | 10 |
| LOF vs. LSTM Autoencoder | +0.137 | 0.0020 | 10 |

*Table 5. Paired Wilcoxon tests on overall AUROC differences, using the same 10 split seeds.*

## 8. Recommendation and Rationale

My final recommendation is operating-regime specific. For accuracy-maximizing analysis, I recommend LOF as the default method for offline review, fleet-health triage, and analyst dashboards. This recommendation is supported by the best overall AUROC, best overall F1, high recall, strong per-fault performance, and statistically significant AUROC differences against Isolation Forest and the LSTM autoencoder.

For latency-constrained edge screening, I would not recommend LOF as the only layer without stress testing. Neighbor-based scoring can become heavier as the dataset scales from the 5-unit sample to the full 50-unit, 30-day configuration. In that regime, Isolation Forest can still be useful as a first-pass screen, but only if paired with rule-based checks for severe battery sag, GPS instability, stale telemetry, and task-success degradation.

For high-stakes escalation, I recommend a hybrid path: LOF as the main anomaly ranking layer, rule-based safety checks for obvious failure modes, and One-Class SVM retained as a challenger model. One-Class SVM is especially worth retaining because it was very close to LOF overall and slightly stronger or effectively tied for software-hang-like faults. This gives the team a second model to compare when moving from synthetic faults to larger or more realistic datasets.

## 9. Assumptions, Limitations, and Caveats

- Synthetic data: the dataset is useful for controlled comparison, but it is not real field telemetry and does not prove deployment performance.
- Controlled faults: injected anomalies are well documented, but real faults may be messier, shorter, correlated, or partially observable.
- Window level: the benchmark evaluates one-minute windows, so it does not measure sub-second alert latency.
- Thresholding: precision, recall, and F1 depend on a fixed contamination-derived threshold that should be recalibrated before field use.
- Scale: LOF should be stress-tested on the full 50-unit, 30-day dataset before final deployment claims.
- Model tuning: the LSTM autoencoder was intentionally compact for local reproducibility. A larger sequence model might perform differently but would require a separate tuning plan.

## 10. Course Correction for Phases C-D

The main course correction for the second half is to keep the same evidence discipline while moving into more abstract tasks. For Week 5, the PPO/SAC reinforcement learning benchmarks

should be framed as analyst exercises that produce transferable lessons about stability, sample efficiency, and coordination, not as direct claims about fielded robots. For Week 6, the AMDC evaluation harness should prioritize clear scenario contracts, scoring consistency, and reliability analysis. For Week 7, the dashboard should focus on decision usefulness rather than visual complexity. For Week 8, the capstone should preserve the evidence chain from market landscape to telemetry, anomaly detection, RL lessons, decision evaluation, and dashboard design.

The key improvement requested after the first midpoint report was to make the empirical evidence more visible. This revised report therefore foregrounds dataset size, fault design, benchmark metrics, per-fault results, and the statistical basis for the final recommendation. That stronger quantitative layer should make the midpoint review easier to score against technical depth, analytical reasoning, reproducibility, communication, initiative, and product anchoring.

## Appendix: Artifact Trail

| Week | Artifacts |
|---|---|
| Week 1 | W01_PhysicalAI_Landscape_Brief.md; W01_PIC20_Conceptual_Map.md; W01_env_check.ipynb |
| Week 2 | W02_Competitive_Matrix.xlsx; W02_Strategic_Gap_Memo.md |
| Week 3 | W03_Telemetry_Generator.py; W03_Telemetry_EDA.ipynb; W03_Feature_Dictionary.md |
| Week 4 | W04_Anomaly_Benchmark.ipynb; W04_Method_Recommendation_Memo.md; W04_MidPoint_Deck.pptx; benchmark result CSVs |

*Table 6. Midpoint artifact trail for Weeks 1-4.*